TEMPLATE WEB FINAL — CSE 4165

Swaron | ID: 0112230418

1. JAVASCRIPT

```
// Basic Output & Reading Inputs:
let output = "";
output += "Average = " + result + "<br>";
document.getElementById("output").innerHTML = output;
let name = document.getElementById("name").value; // [reading from an input]
let marks = Number(document.getElementById("marks").value);
```

```
// I have an array. Some STLs (Array Methods):
.length, .push("") [adds to end], .includes(""), .sort(), .join(", "),
.pop() [removes from end], .unshift("") [adds to the beginning],
.shift() [removes from the beginning], .indexOf(""), .reverse()
```

```
// JavaScript Objects:
let student = {
    name: "Swaron",
    id: "0112230418",
    cgpa: 3.63
};
// Properties can be accessed in 2 ways:
let sName = student.name;
// or
let sId = student["id"];

// JSON Conversion:
// 1. Convert JS Object -> JSON String
let jsonString = JSON.stringify(student);
// 2. Convert JSON String -> Back to JS Object
let parsedStudent = JSON.parse(jsonString);
// Now you can access parsedStudent.name or parsedStudent.cgpa!
```

```
// Q1 Minimal Exam Tracker (String Accumulator + Array Way):
let total = 0, count = 0;
let items = ""; // String way (Just like count++ without array!)
// Or array way: const list = [];

function add() {
    let name = document.getElementById("a").value;
    let val = Number(document.getElementById("b").value);
    if (!name || isNaN(val) || val <= 0) return alert("Invalid");

    count++; total += val;
    items += name + ", "; // Or: list.push(name);

    let avg = total / count;
    let msg = (total <= 5) ? "Low" : (total <= 12) ? "Good" : "Goal reached!";
    if (count > 7 && total < 20) msg = "Increase study time / frequent snacks!";

    document.getElementById("res").innerHTML =
        "Total: " + total + "<br>" +
        "Average: " + avg.toFixed(2) + "<br>" +
        "Items: [" + items.slice(0, -2) + "]<br>" + // Or: list.join(", ")
        "Status: " + msg;

    document.getElementById("a").value = ""; document.getElementById("b").value = "";
}
```

```
// Password Strength Scorer (Regex Checks & Attempt Warning):
let attempts = 0;
function checkPass() {
    let p = document.getElementById("a").value; attempts++; let score = 0;
    if (p.length >= 6) score += Math.floor((p.length - 6) / 2) * 10;
    if (/[A-Z]/.test(p)) score += 15; if (/[a-z]/.test(p)) score += 15;
    if (/[0-9]/.test(p)) score += 20; if (/[!@#$%^&*]/.test(p)) score += 25;
    let lvl = (score <= 30) ? "Very Weak" : (score <= 50) ? "Weak" :
              (score <= 70) ? "Medium" : (score <= 90) ? "Strong" : "Very Strong";
    let msg = (score >= 100) ? "Perfect Password! Success!" : "Score: " + score + " - " +
lvl;
    if (attempts > 8 && score < 71) msg += " | Need practice! Add uppercase & symbols.";
    document.getElementById("res").innerHTML = msg;
}
```

2. HTML & DOM

```
<!-- Text input (for Subject Name, Password, etc.) -->
<input type="text" id="subjectInput" placeholder="Enter Subject">
<!-- Number input (for Study Hours, Calories, Heart Rate, etc.) -->
<input type="number" id="hoursInput" placeholder="Enter Hours">
<!-- Button & Output Div & Script Connection -->
<button onclick="startExam()">Submit</button>
<div id="output"></div>
<script src="script.js"></script>
```

```
// IMPORTANT RULE:
// For normal tags like <div>, <p>, <label>, <h3> (which have an opening
// and closing tag), you use .innerHTML or .innerText.
// For <input> boxes, you NEVER use .innerHTML. You use .value!
```

```
// Reading text input (String) -> just use .value
let subject = document.getElementById("subjectInput").value;
// Reading number input (Int or Float) -> wrap in Number()!
let hours = Number(document.getElementById("hoursInput").value);
// Reading text from a normal div:
<div id="myDiv">Hello World</div>
const value = document.getElementById("myDiv").textContent;
```

```
// JS Math Helpers:
// In JS, bmi.toFixed(2) rounds any number to 2 decimal places.
// Absolute Value: Math.abs(x)   | Rounding Down: Math.floor(x)   | Rounding Up: Math.ceil
(x)
// Math.random() gives a decimal between 0 and 0.999...
// Random integer between min and max: Math.floor(Math.random() * (max - min + 1)) + min;
```

```
<!-- 1. Dropdown (<select>) -->
<select id="location">
    <option value="">Select Location</option>
    <option value="Dhaka">Dhaka</option>
    <option value="Chittagong">Chittagong</option>
</select>
<!-- 2. Radio Buttons (Give both the SAME name="area" so user can't pick both!) -->
<input type="radio" name="area" id="urban" value="Urban"> Urban
<input type="radio" name="area" id="rural" value="Rural"> Rural
<!-- 3. Checkbox --> <input type="checkbox" id="agree" value="yes"> Agree

// Reading Dropdown, Radio Buttons & Disabling Input:
let loc = document.getElementById("location").value;
let isUrban = document.getElementById("urban").checked;
let isRural = document.getElementById("rural").checked;
let isAgree = document.getElementById("agree").checked;
document.getElementById("guessInput").disabled = true;
document.getElementById("output").style.color = "red"; // Dynamic color
```

```
// Patient Vitals Formula (Heart Rate & SpO2 with reduce averages):
const listA = [], listB = [];
function evalVitals() {
    let hr = Number(document.getElementById("a").value);
    let spo2 = Number(document.getElementById("b").value);
    listA.push(hr); listB.push(spo2);
    let avgHR = listA.reduce((s, n) => s + n, 0) / listA.length;
    let score = (100 - spo2) * 2 + Math.abs(hr - 80) * 0.5;
    let st = (score <= 10) ? "SAFE" : (score <= 20) ? "WARNING" : "DANGER";
    document.getElementById("res").innerHTML = "Avg HR: " + avgHR.toFixed(1) + " | Score:
" + score.toFixed(1) + " | Status: " + st;
}
```

TEMPLATE WEB FINAL — CSE 4165 · PHP & SQL

Swaron | ID: 0112230418

3. PHP

```
// Browser URLs:
http://localhost/web_final/rover.php   |   http://localhost/phpmyadmin/
```

```
<?php
$attendees = 150;
$capacity = 60;
$name = "Swaron";
?>
<!-- HTML Form for PHP (Only name attributes needed!) -->
<form action="process.php" method="POST">
    <input type="number" name="students" placeholder="Number of Students">
    <input type="number" name="slicesPerStudent" placeholder="Slices per Student">
    <input type="number" name="slicesPerPizza" placeholder="Slices per Pizza">
    <button type="submit">Calculate</button>
</form>
<?php
// Grab the 3 inputs from the HTML form using their 'name' attribute
$students = $_POST["students"];
$slicesPerStudent = $_POST["slicesPerStudent"];
$slicesPerPizza = $_POST["slicesPerPizza"];

// Useful PHP Math Functions:
// min($a, $b, $c)  -> finds minimum   | max($a, $b, $c)  -> finds maximum
// ceil($a / $b)    -> rounds UP to next whole integer
// (int)($a / $b)   -> integer division (for digit extraction loop)
// abs($a - $b)     -> absolute positive difference

// PHP Array & Loop:
$fruits = ["Apple", "Banana", "Mango", "Orange"];
for ($i = 0; $i < count($fruits); $i++) {
    echo $fruits[$i] . "<br>";
}
?>
```

```
// Pattern 2A: Ceiling Allocation & Waste (Direct Flat Code)
$demand = floatval($_POST['a']); $rate = floatval($_POST['b']); $cap = floatval($_POST
['c']);
$unitsNeeded = ceil($demand / $cap); // Always round UP!
$totalCap    = $unitsNeeded * $cap;
$empty       = $totalCap - $demand;
$wastedCost  = $empty * $rate;
$unusedPct   = ($totalCap > 0) ? ($empty / $totalCap) * 100 : 0;
$status      = ($unusedPct <= 10) ? "Efficient" : (($unusedPct <= 25) ? "Acceptable" : "Wa
steful");
echo "Units: $unitsNeeded | Empty: $empty | Wasted: $wastedCost BDT | Status: $status";
```

```
// Pattern 2B: Credits / Target Comparison & Category
$total = floatval($_POST['cred']) * floatval($_POST['num']);
$target = floatval($_POST['tar']);
$cat = ($total >= 120) ? "Excellent" : (($total >= 90) ? "Good" : "Average");
// In PHP string math, always wrap subtraction in parentheses!
if ($total == $target)    $st = "At target (0 diff)";
elseif ($total > $target) $st = "Above target by " . ($total - $target);
else                      $st = "Below target by " . ($target - $total);
echo "Total: $total | Cat: $cat | Target: $st";
```

ZERO-CRASH EMERGENCY RULES & GOTCHAS

- **WHERE vs HAVING:** WHERE filters rows; HAVING filters groups. Never put `COUNT(*)` or `AVG()` in WHERE!
- **SQL Order:** `WHERE > GROUP BY > HAVING > ORDER BY`.
- **Execution Order:** Always run tasks in exact sequence. Early updates affect later queries.
- **SQL Quotes:** Text in single quotes `'Text'`. Double apostrophes: `'Cox''s Bazar'`.

EXAM WORDING → EXACT SQL MAPPING

Exam Phrasing:	Exact SQL Command:
"For each category / dept"	`GROUP BY Category` (include in SELECT)
"Total entries in each group"	`SELECT Cat, COUNT(*) FROM t GROUP BY Cat`
"Only groups where avg > 4.0"	`HAVING AVG(Rating) > 4.0`
"Sorted highest to lowest"	`ORDER BY col_name DESC`
"Contains word 'Beach'"	`WHERE Title LIKE '%Beach%'`
"Between 4.0 and 4.5"	`WHERE Rating BETWEEN 4.0 AND 4.5`
"Total store net worth"	`SELECT SUM(Price * Stock) AS NetWorth FROM items`
"Bonus with cap ≤ 60k"	`UPDATE t SET P=P+500 WHERE P>300 AND (P+500)<=60000`

4. SQL & PHP MYSQLI

```
-- 1. Create and select the database
CREATE DATABASE IF NOT EXISTS uiu_bookshop;
USE uiu_bookshop;
-- 2. Create the unified table (INT, VARCHAR, DOUBLE, DATE)
DROP TABLE IF EXISTS book_info;
CREATE TABLE book_info (
    BookID INT PRIMARY KEY, BookTitle VARCHAR(100), Author VARCHAR(100),
    Category VARCHAR(100), Price DOUBLE, Stock INT, Rating DOUBLE, Status VARCHAR(50)
);
-- 3. Insert all rows (Escape apostrophe by doubling: 'Cox''s Bazar')
INSERT INTO book_info VALUES
(1, 'Web Basics', 'John Smith', 'Programming', 450.00, 10, 4.8, 'Active'),
(2, 'Database Guide', 'Alice Roy', 'Programming', 550.00, 8, 4.2, 'Active'),
(3, 'English Grammar', 'Mary Khan', 'Language', 300.00, 15, 4.5, 'Pending'),
(4, 'Business Math', 'David Lee', 'Business', 400.00, 12, 4.1, 'Active'),
(5, 'Python Mastery', 'Sara Ahmed', 'Programming', 600.00, 5, 3.9, 'Returned'),
(6, 'Cox''s Pen', 'Paul Costa', 'Language', 350.00, 7, 4.6, 'Returned');
```

```
// Default MySQL settings: Host: "localhost" | User: "root" | Password: "" (empty)
<?php
$conn = new mysqli("localhost", "root", "", "uiu_bookshop");
if ($conn->connect_error) die("Connection Failed: " . $conn->connect_error);

// Direct While Loop Print (Fast marks, no bloat):
$res = $conn->query("SELECT Category, COUNT(*) AS Total, AVG(Price) AS AvgP FROM book_info
GROUP BY Category");
while ($row = $res->fetch_assoc()) {
    echo $row['Category'] . " - " . $row['Total'] . " - " . $row['AvgP'] . "<br>";
}
?>
```

ALL 15 SQL QUERY ARCHETYPES (TAGGED WITH PAST PAPERS 242–261)

#	Type & Past	Exact SQL Command (Runs on book_info / items table)
1	Count per group 243 251 253 261	`SELECT Category, COUNT(*) AS Total FROM book_info GROUP BY Category;`
2	Total & avg per group 242 252	`SELECT Category, SUM(Price) AS Revenue, AVG(Price) AS AvgPrice FROM book_info GROUP BY Category;`
3	Groups with > 1 row 253	`SELECT Status, COUNT(*) AS Total FROM book_info GROUP BY Status HAVING COUNT(*) > 1;`
4	Top 2 days by date 242	`SELECT SaleDate, SUM(Price) AS Rev FROM book_info WHERE SaleDate BETWEEN '2024-01-01' AND '2024-01-31' GROUP BY SaleDate ORDER BY Rev DESC LIMIT 2;`
5	LIKE with OR 261	`SELECT BookTitle, Category, Rating FROM book_info WHERE Rating > 4.5 OR BookTitle LIKE '%Pen%' ORDER BY Rating DESC;`
6	AND filter 261	`SELECT BookTitle, Price FROM book_info WHERE Price > 400 AND Category = 'Programming';`
7	Net Worth (Total) 261	`SELECT SUM(Price * Stock) AS NetWorth FROM book_info;`
8	HAVING on Avg 261	`SELECT Category, COUNT(*) AS Items, AVG(Rating) AS AvgR, SUM(Stock) AS TotStk FROM book_info GROUP BY Category HAVING AVG(Rating) > 4.4 ORDER BY AvgR DESC;`
9	Calculated HAVING 261	`SELECT Category, SUM(Price * Stock) AS Val FROM book_info GROUP BY Category HAVING SUM(Price * Stock) > 10000 ORDER BY Val DESC;`
10	Distinct dates count 242	`SELECT Category, AVG(Stock) FROM book_info GROUP BY Category HAVING COUNT(DISTINCT SaleDate) >= 2;`
11	Relative avg (Top Seller) 252	`SELECT b.BookTitle, b.Price, CASE WHEN b.Price > (SELECT AVG(b2.Price) FROM book_info b2 WHERE b2.Category = b.Category) THEN 'Top Seller' ELSE 'Regular Seller' END AS Status FROM book_info b;`
12	UPDATE with <> 243 251 252 253	`UPDATE book_info SET Status = 'Low' WHERE Price < 400 AND Status <> 'Returned';`
13	Bonus with ceiling cap 243 251	`UPDATE book_info SET Price = Price + 50 WHERE Price > 300 AND (Price + 50) <= 600;`
14	10% discount / raise 252 253 261	`UPDATE book_info SET Price = Price * 0.90 WHERE Category = 'Language'; -- raise: * 1.10`
15	Update 2 cols at once 261	`UPDATE book_info SET Rating = Rating + 0.2, Price = Price * 1.10 WHERE Rating BETWEEN 4.0 AND 4.5 AND Price > 0;`